

LAPORAN PRAKTIKUM

**TUGAS 5 Database Container: PostgreSQL, pgAdmin, Volume, Backup,
dan Restore**

Dosen Pengampu : Ferry Astika Saputra



Disusun Oleh :

Adam Rasyid Nurmuhammad

3126640025

D4-IT-B

**PROGRAM STUDI SARJANA TERAPAN
JURUSAN TEKNIK INFORMATIKA
DEPARTEMEN TEKNIK INFORMATIKA DAN KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2026**

BAB I PENDAHULUAN.....	3
1.1 Latar Belakang	3
BAB II LANDASAN TEORI	4
2.1 Konsep Backup dan Restore Database PostgreSQL dengan Docker.....	4
2.2 Peta Konsep: Bagaimana cara kerja persistensi data dengan Docker volumes?	4
2.3 Peta Konsep: Backup dan Restore PostgreSQL dengan Docker	6
BAB III Pengerjaan Praktikum.....	7
3.1 Persiapan Direktori Bab 5	7
3.2 Pembuatan File Konfigurasi Laboratorium	7
3.3 Permission dan Validasi File	10
3.4 Menjalankan Stack dan Akses pgAdmin	11
3.5 Pengujian Schema dan Data Awal	11
3.6 Pengujian Persistensi Volume Docker	12
3.7 Prosedur Pembuatan Backup dan Checksum.....	12
3.8 Prosedur Disaster Recovery dan Restore Test	12
3.9 Monitoring Dasar Container dan Volume	13
3.10 Prosedur Cleanup Database Uji dan Stack.....	13
BAB IV HASIL DAN PEMBAHASAN.....	14
4.1 Evaluasi dan Latihan Mandiri.....	14
4.2 Analisis Wajib	14
BAB V PENUTUP.....	16
5.1 Kesimpulan	16
5.2 Catatan Penggunaan AI.....	16

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam era arsitektur cloud-native dan ekosistem microservices modern, kontainerisasi menggunakan Docker telah menjadi bagian penting dalam aplikasi karena menjamin portabilitas, isolasi sumber daya, dan kemudahan deployment. Segala modifikasi berkas yang dilakukan di dalam container layer akan down saat siklus hidup kontainer berakhir (restart, update, atau destroy). Karakteristik ini menjadi tantangan besar ketika mengoperasikan sistem (RDBMS) seperti PostgreSQL yang bersifat stateful dan menuntut penyimpanan data secara permanen.

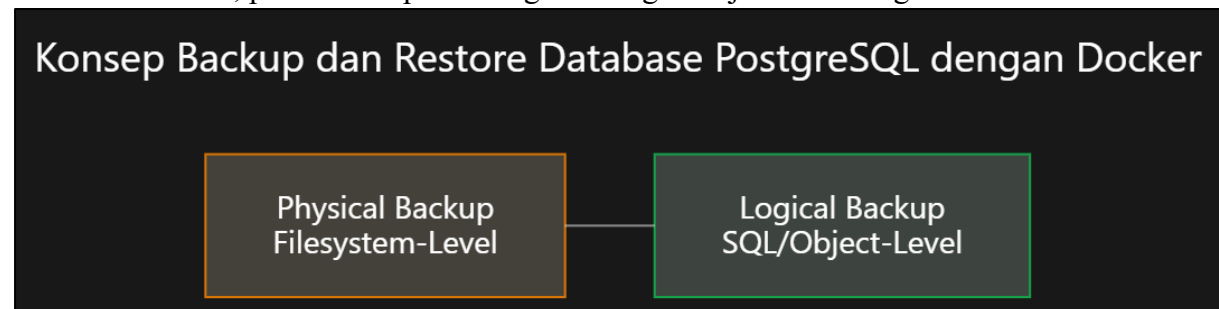
Untuk mengatasi keterbatasan tersebut, Docker menyediakan mekanisme persistensi penyimpanan eksternal, khususnya Docker Named Volumes. Melalui named volume, direktori basis data (/var/lib/postgresql/data) dipindahkan ke area penyimpanan host yang dikelola oleh Docker Engine. Hal ini memungkinkan data tetap persisten melintasi siklus hidup kontainer. Namun demikian, persistensi volume saja belum mencukupi standar ketahanan DevSecOps. Kerusakan disk, serangan ransomware, korupsi data, hingga human error tetap dapat merusak sistem. Oleh karena itu, prosedur pencadangan (backup) berbasis `pg_dump`, verifikasi keaslian kriptografis SHA-256, menjadi keharusan mutlak Pembangunan system berkelanjutan

BAB II

LANDASAN TEORI

2.1 Konsep Backup dan Restore Database PostgreSQL dengan Docker

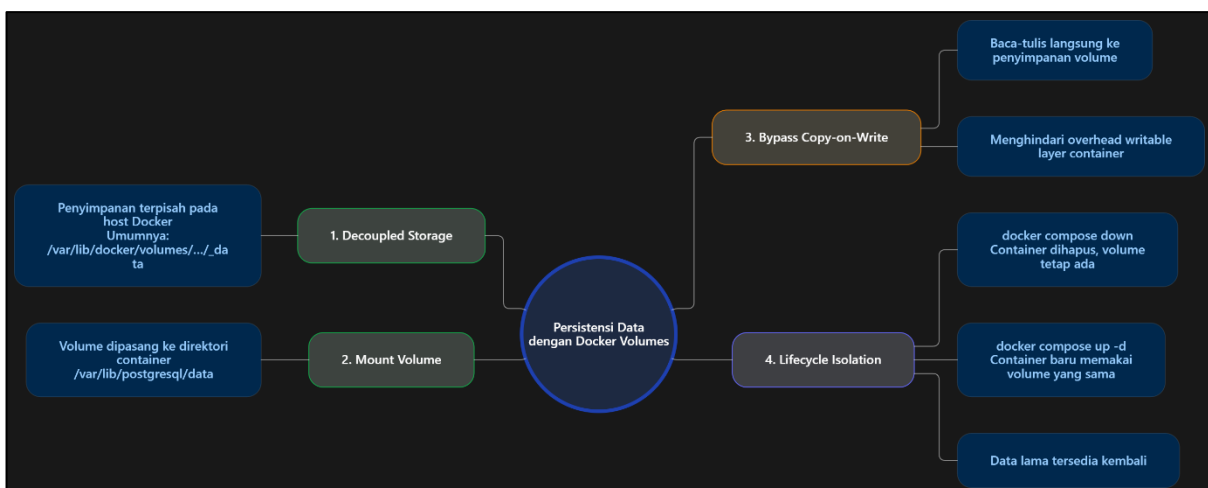
Pencadangan (backup) merupakan tindakan menyalin kondisi basis data ke media penyimpanan terpisah, sedangkan pemulihan (restore) merupakan rekonstruksi basis data ke status operasional konsisten pasca terjadinya insiden. Pada PostgreSQL yang berjalan di kontainer Docker, pendekatan pencadangan terbagi menjadi dua kategori utama:



1. Physical Backup (Filesystem-Level): Menyalin seluruh direktori fisik \$PGDATA secara biner. Kelemahan pendekatan ini adalah ukuran file sangat besar, terikat kaku pada arsitektur OS dan versi mayor PostgreSQL yang identik, serta memerlukan WAL archiving atau penghentian database agar tidak corrupt.

2. Logical Backup (SQL/Object-Level): Mengekstrak skema DDL dan data DML menggunakan utilitas bawaan PostgreSQL `pg_dump`. Mekanisme ini membaca snapshot transaksional secara non-blocking (zero downtime).

2.2 Peta Konsep: Bagaimana cara kerja persistensi data dengan Docker volumes?



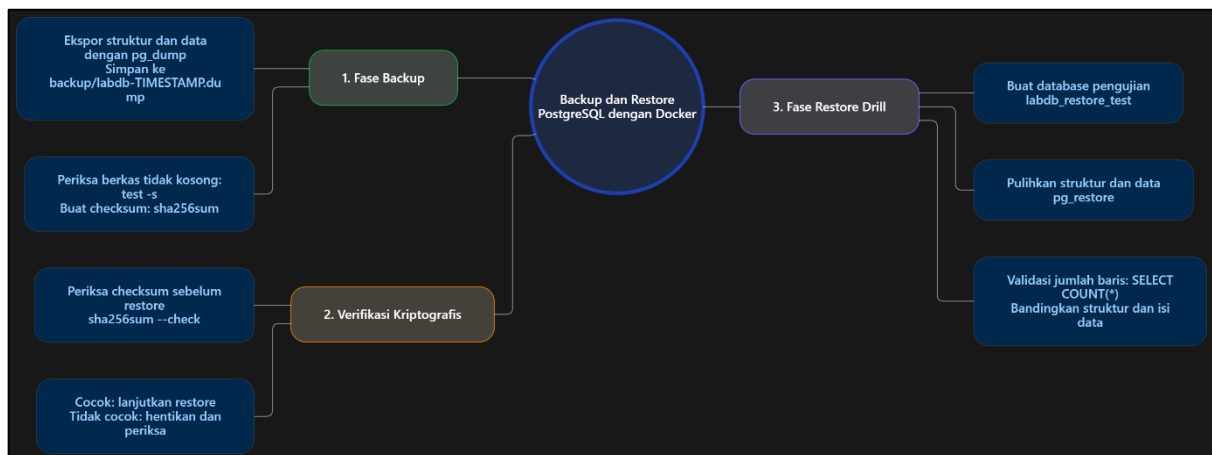
1. Decoupled Storage: Docker membuat direktori penyimpanan khusus pada filesystem host di `/var/lib/docker/volumes/<volume_name>/_data`.

2. VFS Namespace Mount: Engine Docker me-mount direktori host tersebut langsung ke direktori target di dalam kontainer (`/var/lib/postgresql/data`).

3. Bypass Copy-on-Write: Operasi tulis-baca langsung dialirkan ke disk host tanpa melalui overhead layer kontainer, menghasilkan performa I/O native.

4. Lifecycle Isolation: Saat kontainer dihentikan atau dihancurkan (docker compose down), volume tetap dipertahankan. Saat kontainer dinyalakan kembali, kontainer baru langsung mengaitkan kembali volume yang sama.

2.3 Peta Konsep: Backup dan Restore PostgreSQL dengan Docker



1. Fase Backup: Perintah `docker compose exec -T postgres-db pg_dump` mengalirkan data biner secara streaming keluar kontainer langsung ke berkas host `backup/labdb-TIMESTAMP.dump`. Host memvalidasi bahwa berkas tidak nol byte (`test -s`) dan segera menghitung digest SHA-256 (`sha256sum`).
2. Fase Verifikasi Kriptografis: Sebelum restorasi dimulai, `sha256sum --check` dijalankan untuk memastikan berkas terbebas dari silent bit rot atau perubahan tidak sah.
3. Fase Restore Drill: Database baru `labdb_restore_test` dibuat pada kontainer, utilitas `pg_restore` mengeksekusi rekonstruksi struktur dan record data, lalu kueri validasi `SELECT COUNT(*)` memastikan seluruh data pulih sempurna 100%.

BAB III

Pengerjaan Praktikum

3.1 Persiapan Direktori Bab 5

Menyiapkan tata kelola direktori kerja proyek laboratorium agar struktur berkas terpisah secara rapi:

```
adam@Devsecops:~$ mkdir -p ~/docker-lab/bab-5/{init,backup,pgadmin,scripts}
adam@Devsecops:~$ cd ~/docker-lab/bab-5
adam@Devsecops:~/docker-lab/bab-5$ ls
backup  init  pgadmin  scripts
```

3.2 Pembuatan File Konfigurasi Laboratorium

A. Script 1 — File .env.example (Template Konfigurasi Lingkungan):

```
adam@Devsecops:~/docker-lab/bab-5$ nano .env.example
adam@Devsecops:~/docker-lab/bab-5$ cp .env.example .env
adam@Devsecops:~/docker-lab/bab-5$ chmod 600 .env
```

B. Script 2 — File .gitignore (Pencegahan Kebocoran Kredensial dan Dump):

```
GNU nano 6.2 .gitignore
.env
backup/*.dump
backup/*.sha256
backup/*.log
```

C. Script 3 — File compose.yaml (Multi-Container Stack PostgreSQL & pgAdmin):

```

GNU nano 6.2                                     compose.yaml *
services:
  postgres-db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    ports:
      - "127.0.0.1:5432:5432"
    volumes:
      - pg-data:/var/lib/postgresql/data
      - ./init:/docker-entrypoint-initdb.d:ro
    networks:
      - data-net
    healthcheck:
      test:
        - CMD-SHELL
        - "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"
      interval: 5s
      timeout: 5s
      retries: 10
      start_period: 10s
      restart: unless-stopped

  pgadmin:
    image: dpage/pgadmin4:latest
    environment:
      PGADMIN_DEFAULT_EMAIL: ${PGADMIN_DEFAULT_EMAIL}
      PGADMIN_DEFAULT_PASSWORD: ${PGADMIN_DEFAULT_PASSWORD}
    ports:
      - "127.0.0.1:5050:80"
    volumes:
      - pgadmin-data:/var/lib/pgadmin
      - ./pgadmin/servers.json:/pgadmin4/servers.json:ro

```

D. Script 4 — File init/01-schema.sql (Inisialisasi Tabel dan Data Awal Mahasiswa):

```

GNU nano 6.2                                     init/01-schema.sql
CREATE TABLE IF NOT EXISTS students (
  id BIGSERIAL PRIMARY KEY,
  nrp VARCHAR(20) UNIQUE NOT NULL,
  name TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO students (nrp, name)
VALUES
  ('31230001', 'Mahasiswa Satu'),
  ('31230002', 'Mahasiswa Dua')
ON CONFLICT (nrp) DO NOTHING;

CREATE INDEX IF NOT EXISTS idx_students_name
ON students (name);

```

E. Script 5 — File pgadmin/servers.json (Pendaftaran Server Otomatis pada pgAdmin):

```

GNU nano 6.2                                pgadmin/servers.json
{
  "Servers": {
    "1": {
      "Name": "PostgreSQL Bab 5",
      "Group": "Laboratorium DevSecOps",
      "Host": "postgres-db",
      "Port": 5432,
      "MaintenanceDB": "labdb",
      "Username": "labuser",
      "SSLMode": "prefer"
    }
  }
}

```

F. Script 6 — File scripts/backup.sh (Otomasi Logical Backup dan SHA-256 Digest):

```

GNU nano 6.2                                scripts/backup.sh
set -Eeuo pipefail

cd "$(dirname "$0")/.."

if [[ ! -f .env ]]; then
  echo "[FAIL] File .env tidak ditemukan." >&2
  exit 1
fi

set -as
source .env
set +a

mkdir -p backup

timestamp="$(date -u +%Y%m%dT%H%M%SZ)"
dump_file="backup/${POSTGRES_DB}-${timestamp}.dump"
checksum_file="${dump_file}.sha256"

docker compose exec -T postgres-db \
  pg_dump \
    --username "$POSTGRES_USER" \
    --dbname "$POSTGRES_DB" \
    --format custom \
    --no-owner \
    --no-privileges \
    > "$dump_file"

test -s "$dump_file"
sha256sum "$dump_file" > "$checksum_file"

echo "[PASS] Backup: $dump_file"
echo "[PASS] Checksum: $checksum_file"

```

G. Script 7 — File scripts/restore-test.sh (Otomasi Uji Pemulihan Bencana):

```

GNU nano 6.2 scripts/restore-test.sh
#!/usr/bin/env bash
set -Eeuo pipefail

cd "$(dirname "$0")/.."

if [[ ! -f .env ]]; then
    echo "[FAIL] File .env tidak ditemukan." >&2
    exit 1
fi

set -a
source .env
set +a

dump_file="${1:-}"
restore_database="${POSTGRES_DB}_restore_test"

if [[ -z "$dump_file" || ! -s "$dump_file" ]]; then
    echo "Penggunaan: $0 backup/nama-file.dump" >&2
    exit 1
fi

if [[ -f "${dump_file}.sha256" ]]; then
    sha256sum --check "${dump_file}.sha256"
fi

docker compose exec -T postgres-db \
    dropdb --username "$POSTGRES_USER" --if-exists "$restore_database"

docker compose exec -T postgres-db \
    createdb --username "$POSTGRES_USER" "$restore_database"

docker compose exec -T postgres-db \
    pg_restore \

```

3.3 Permission dan Validasi File

Memberikan hak akses eksekusi pada skrip Bash dan memvalidasi konfigurasi Docker Compose:

```

adam@Devsecops:~/docker-lab/bab-5$ chmod +x scripts/backup.sh scripts/restore-test.sh
adam@Devsecops:~/docker-lab/bab-5$ find . -maxdepth 3 -type f | sort
./compose.yaml
./env
./env.example
./gitignore
./init/01-schema.sql
./pgadmin/servers.json
./scripts/backup.sh
./scripts/restore-test.sh
adam@Devsecops:~/docker-lab/bab-5$ docker compose config
name: bab-5
services:
  pgadmin:
    depends_on:
      postgres-db:
        condition: service_healthy
        required: true
    environment:
      PGADMIN_DEFAULT_EMAIL: admin@example.local
      PGADMIN_DEFAULT_PASSWORD: admin123
    image: dpage/pgadmin4:latest
    networks:
      data-net: null

```

```

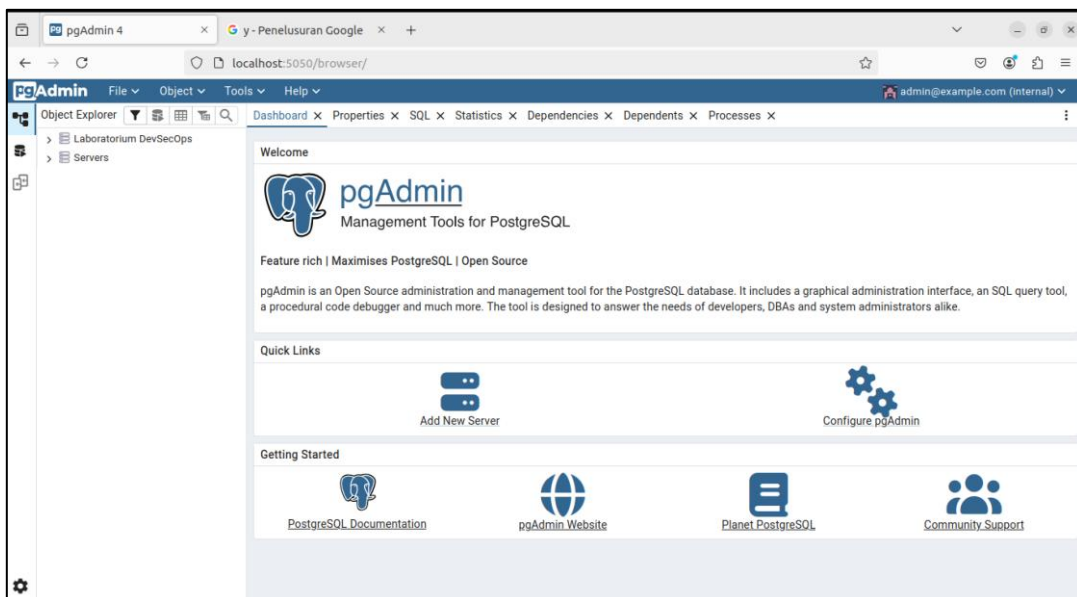
adam@Devsecops:~/docker-lab/bab-5$ docker compose config --services
postgres-db
pgadmin

```

3.4 Menjalankan Stack dan Akses pgAdmin

Menjalankan stack secara background, memantau hingga healthy, dan mengakses dashboard pgAdmin pada browser:

```
adam@Devsecops:~/docker-lab/bab-5$ docker compose up -d
[+] up 22/22
✓ Image dpape/pgadmin4:latest Pulled
✓ Network bab-5_data-net Created
✓ Volume bab-5_pgadmin-data Created
✓ Volume bab-5_pg-data Created
✓ Container bab-5-postgres-db-1 Healthy
✓ Container bab-5-pgadmin-1 Started
adam@Devsecops:~/docker-lab/bab-5$ docker compose ps
NAME                IMAGE                COMMAND                SERVICE
bab-5-pgadmin-1     dpape/pgadmin4:latest "/entrypoint.sh"      pgadmin
bab-5-postgres-db-1 postgres:16-alpine   "docker-entrypoint.s..." postgres-d
32/tcp
adam@Devsecops:~/docker-lab/bab-5$ docker compose logs --tail 100
postgres-db-1 | The files belonging to this database system will be owned by user
postgres-db-1 | This user must also own the server process.
postgres-db-1 |
postgres-db-1 | The database cluster will be initialized with locale "en_US.utf8"
postgres-db-1 | The default database encoding has accordingly been set to "UTF8"
postgres-db-1 | The default text search configuration will be set to "english".
```



3.5 Pengujian Schema dan Data Awal

Menguji tabel students dan dua record data awal hasil inisialisasi volume:

```
adam@Devsecops:~/docker-lab/bab-5$ docker compose exec -T postgres-db \
psql -U labuser -d labdb \
-c "\dt"
      List of relations
Schema |   Name   | Type | Owner
-----+-----+-----+-----
public | students | table | labuser
(1 row)

adam@Devsecops:~/docker-lab/bab-5$ docker compose exec -T postgres-db \
psql -U labuser -d labdb \
-c "SELECT id, nrp, name, created_at FROM students ORDER BY id;"
 id | nrp      | name           | created_at
-----+-----+-----+-----
  1 | 31230001 | Mahasiswa Satu | 2026-10-08 10:31:46.170756+00
  2 | 31230002 | Mahasiswa Dua  | 2026-10-08 10:31:46.170756+00
(2 rows)
```

3.6 Pengujian Persistensi Volume Docker

Menyisipkan record ketiga (Mahasiswa Tiga), merecreate container dengan down lalu up -d, membuktikan data tetap persisten:

```
adam@Devsecops:~/docker-lab/bab-5$ docker compose exec -T postgres-db \
psql -U labuser -d labdb \
-c "INSERT INTO students(nrp, name) VALUES ('31230003', 'Mahasiswa Tiga') ON CONFLICT DO NOTHING;"
INSERT 0 1
adam@Devsecops:~/docker-lab/bab-5$ docker compose down
docker compose up -d
docker compose exec -T postgres-db \
psql -U labuser -d labdb \
-c "SELECT nrp, name FROM students ORDER BY nrp;"
[+] down 3/3
✓ Container bab-5-pgadmin-1      Removed
✓ Container bab-5-postgres-db-1  Removed
✓ Network bab-5_data-net         Removed
[+] up 3/3
✓ Network bab-5_data-net         Created
✓ Container bab-5-postgres-db-1  Healthy
✓ Container bab-5-pgadmin-1      Started
 nrp | name
-----+-----
31230001 | Mahasiswa Satu
31230002 | Mahasiswa Dua
31230003 | Mahasiswa Tiga
(3 rows)
```

3.7 Prosedur Pembuatan Backup dan Checksum

Mengeksekusi skrip backup.sh, menghasilkan file dump biner terkompresi dan memverifikasi checksum SHA-256 (OK):

```
adam@Devsecops:~/docker-lab/bab-5$ ./scripts/backup.sh
[PASS] Backup: backup/labdb-20261008T125633Z.dump
[PASS] Checksum: backup/labdb-20261008T125633Z.dump.sha256
adam@Devsecops:~/docker-lab/bab-5$ ls -lh backup/
total 8.0K
-rw-rw-r-- 1 adam adam 3.4K Oct  8 19:56 labdb-20261008T125633Z.dump
-rw-rw-r-- 1 adam adam 101 Oct  8 19:56 labdb-20261008T125633Z.dump.sha256
adam@Devsecops:~/docker-lab/bab-5$ sha256sum --check backup/*.dump.sha256
backup/labdb-20261008T125633Z.dump: OK
```

3.8 Prosedur Disaster Recovery dan Restore Test

Menjalankan skrip restore-test.sh ke basis data uji labdb_restore_test dan mengonfirmasi integritas data mahasiswa:

```
adam@Devsecops:~/docker-lab/bab-5$ latest_dump="$(find backup -maxdepth 1 -name '*.dump' -type f | sort | tail -n 1)"
./scripts/restore-test.sh "$latest_dump"
backup/labdb-20261008T125633Z.dump: OK
NOTICE: database "labdb_restore_test" does not exist, skipping
total_students
-----
          3
(1 row)

[PASS] Restore berhasil ke database: labdb_restore_test
```

```
adam@Devsecops:~/docker-lab/bab-5$ docker compose exec -T postgres-db \
psql -U labuser -d labdb_restore_test \
-c "SELECT nrp, name FROM students ORDER BY nrp;"
nrp      | name
-----+-----
31230001 | Mahasiswa Satu
31230002 | Mahasiswa Dua
31230003 | Mahasiswa Tiga
(3 rows)
```

3.9 Monitoring Dasar Container dan Volume

Melakukan inspeksi resource, log transaksi, pg_isready, serta penggunaan disk volume:

```
adam@Devsecops:~/docker-lab/bab-5$ docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS        PORTS
bab-5-pgadmin-1     dpape/pgadmin4:latest "/entrypoint.sh"       pgadmin    10 minutes ago Up 10 minutes 443/tcp, 127.0.0.1:50
0->80/tcp
bab-5-postgres-db-1 postgres:16-alpine   "docker-entrypoint.s... postgres-db 10 minutes ago Up 10 minutes (healthy) 127.0.0.1:5432->5432/
cp

adam@Devsecops:~/docker-lab/bab-5$ docker compose logs postgres-db --tail 10
postgres-db-1 | 2026-10-08 12:49:14.630 UTC [1] LOG: starting PostgreSQL 16.15 on x86_64-pc-linux-musl, compiled by gcc (Alpine 15.2.0) 15.2.0, 64-
bit
postgres-db-1 | 2026-10-08 12:49:14.631 UTC [1] LOG: listening on IPv4 address "0.0.0.0", port 5432
postgres-db-1 | 2026-10-08 12:49:14.631 UTC [1] LOG: listening on IPv6 address ":::", port 5432
postgres-db-1 | 2026-10-08 12:49:14.642 UTC [1] LOG: listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
postgres-db-1 | 2026-10-08 12:49:14.666 UTC [29] LOG: database system was shut down at 2026-10-08 12:49:11 UTC
postgres-db-1 | 2026-10-08 12:49:14.687 UTC [1] LOG: database system is ready to accept connections
postgres-db-1 | 2026-10-08 12:54:14.765 UTC [27] LOG: checkpoint starting: time
postgres-db-1 | 2026-10-08 12:54:14.922 UTC [27] LOG: checkpoint complete: wrote 4 buffers (0.0%); 0 WAL file(s) added, 0 removed, 0 recycled; writ
e=0.117 s, sync=0.012 s, total=0.158 s; sync files=3, longest=0.006 s, average=0.004 s; distance=0 kB, estimate=0 kB; lsn=0/195C038, redo lsn=0/195CD0
e
postgres-db-1 | 2026-10-08 12:59:15.026 UTC [27] LOG: checkpoint starting: time
adam@Devsecops:~/docker-lab/bab-5$ docker compose exec -T postgres-db pg_isready -U labuser -d labdb
/var/run/postgresql:5432 - accepting connections
adam@Devsecops:~/docker-lab/bab-5$ docker compose exec -T postgres-db psql -U labuser -d labdb -c "SELECT version();"
version
-----
PostgreSQL 16.15 on x86_64-pc-linux-musl, compiled by gcc (Alpine 15.2.0) 15.2.0, 64-bit
(1 row)

adam@Devsecops:~/docker-lab/bab-5$ docker volume ls
DRIVER    VOLUME NAME
local     bab-3_pg-data
local     bab-5_pg-data
local     bab-5_pgadmin-data
local     data-vol

adam@Devsecops:~/docker-lab/bab-5$ docker system df -v
Images space usage:
REPOSITORY    TAG        IMAGE ID        CREATED        SIZE        SHARED SIZE    UNIQUE SIZE    CONTAINERS
bab-4-flask-app latest     8c789867dbbd    7 days ago    200MB        177.2MB        22.87MB        1
bab-3-app     latest     675e158690f8    2 weeks ago    220MB        177.2MB        42.8MB         1
```

3.10 Prosedur Cleanup Database Uji dan Stack

Menghapus database hasil restore test dan mematikan layanan secara aman tanpa menghilangkan volume data:

```
adam@Devsecops:~/docker-lab/bab-5$ docker compose up -d postgres-db
docker compose exec -T postgres-db \
dropdb -U labuser --if-exists labdb_restore_test
[+] up 2/2
✓ Network bab-5_data-net Created
✓ Container bab-5-postgres-db-1 Started

adam@Devsecops:~/docker-lab/bab-5$ docker compose down
[+] down 3/3
✓ Container bab-5-pgadmin-1 Removed
✓ Container bab-5-postgres-db-1 Removed
✓ Network bab-5_data-net Removed
```

BAB IV

HASIL DAN PEMBAHASAN

4.1 Evaluasi dan Latihan Mandiri

1. Mengapa init script tidak dijalankan ulang saat volume lama masih ada?

Jawab:

Saat volume lama masih ada, entripoint mendeteksi bahwa basis data sudah terinisialisasi. Untuk mencegah data corruption atau kegagalan konflik kunci unik, entripoint secara sengaja melewati (skip) direktori inisialisasi tersebut. Untuk pembaruan skema di lingkungan produksi, harus digunakan mekanisme migrasi skema (seperti Flyway atau Prisma).

2. Apa risiko meletakkan password database pada docker-compose.yml?

Jawab:

Risiko meletakkan password pada docker-compose.yml adalah berkas compose dilacak oleh git dapat memungkinkan terunggah ke repositori publik. Metadata environment tersimpan dalam engine Docker dan dapat dibaca oleh siapa pun pengguna lokal via docker inspect. Selanjutnya, Kredensial dapat tercetak pada log automasi pipeline atau terbaca pada /proc/<PID>/environ di host

3. Bagaimana cara membuktikan backup dapat dipulihkan?

Jawab

Backup dapat dibuktikan bisa dipulihkan dengan melakukan uji restore ke database terpisah menggunakan pg_restore. Sebelum pemulihan, integritas file diperiksa melalui checksum SHA-256. Setelah restore selesai tanpa error, coba eksekusi kueri validasi SQL (misalnya SELECT COUNT(*)) untuk memastikan jumlah record dan keutuhan skema identik dengan data sumber.

4. Apa bedanya logical backup pg_dump dan backup filesystem volume mentah?

Jawab

Logical backup menggunakan pg_dump untuk menyalin struktur dan isi database Sementara itu, backup filesystem volume mentah menyalin berkas fisik database secara langsung dan lebih bergantung pada kesesuaian versi serta lingkungan PostgreSQL.

5. Apa dampak docker compose down -v terhadap database?

Penambahan -v (atau --volumes) menginstruksikan Docker untuk menghapus seluruh volume anonim maupun named volumes yang tercantum pada berkas Compose (pg-data dan pgadmin-data). Dampaknya direktori fisik penyimpanan data pada host dihapus secara permanen, seluruh data dan transaksi musnah total. Saat stack dijalankan kembali dengan up -d, Docker akan membuat volume baru yang kosong dari awal. Di lingkungan produksi, ini merupakan bencana fatal kehilangan data.

4.2 Analisis Wajib

1. Bagaimana Menyimpan Kredensial DB Service di Container Secara Aman?

Jawab :

Simpan Kredensial sebagai dokumen yang terenkripsi (tmpfs) pada /run/secrets/db_password, sehingga password tidak pernah muncul di environment variable atau docker inspect. Selanjutnya, memanfaatkan HashiCorp Vault atau Cloud

KMS yang menginjeksi dynamic short-lived credentials dengan rotasi otomatis. Terakhir, Memastikan secret di-chmod 600 dan wajib terdaftar di .gitignore.

2. Risiko Keamanan dan Operasional yang Relevan pada Bab Ini

Jawab:

Perintah `down -v` menghapus data permanen tanpa konfirmasi proteksi.

Berkas cadangan .dump disimpan tanpa enkripsi di host. Jika host disusupi, penyerang langsung memiliki salinan seluruh isi database.

3. Rekomendasi Perbaikan Menuju Lingkungan Produksi

Jawab:

Migrasi dari .env ke Docker Secrets dengan `POSTGRES_PASSWORD_FILE`.

Mengaktifkan SSL/TLS wajib pada PostgreSQL (`sslmode=verify-full`) dan mengenkripsi berkas dump menggunakan AES-256 / GPG.

Automated Recovery Pipeline: Mengintegrasikan `restore-test.sh` ke pipeline CI/CD untuk memvalidasi pemulihan bencana tanpa pengecekan manual.

Membatasi izin pengguna aplikasi hanya pada operasi CRUD tabel tertentu, bukan sebagai superuser.

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan serangkaian pengujian dan implementasi praktikum Database Container: PostgreSQL, pgAdmin, Volume, Backup, dan Restore pada Bab 5, dapat disimpulkan bahwa

1. Named volume pg-data berhasil memisahkan siklus hidup data dari siklus hidup kontainer, sehingga dapat menjaga data tetap utuh.
2. Direktori /docker-entrypoint-initdb.d hanya dieksekusi saat volume kosong, melindungi basis data aktif dari kerusakan skema saat kontainer direstart.
3. Praktikum ini menegaskan pentingnya pembatasan port, sanitasi repositori via .gitignore, serta memberikan alur pengamanan tingkat produksi melalui Docker Secrets.

5.2 Catatan Penggunaan AI

Dalam penyusunan dokumen ini, penulis menggunakan kecerdasan buatan (AI) untuk membantu pembuatan peta konsep, eksplorasi materi, serta penyempurnaan struktur dan bahasa dokumen.